

Debug Tools for Inverted Pendulum

This directory contains three debugging tools for the wheeled inverted pendulum experiment. Each tool consists of:

- `.cpp` **file**: mbed microcontroller program (C++ source code)
- `.bin` **file**: Pre-compiled binary ready to flash to mbed LPC1768
- `.pde` **file**: Processing visualization program (runs on PC)

Tools Overview

1. Motor Controller (`motor_controller/`)

Purpose: Test motor driver circuit and control motor interactively

mbed Program (`motor_controller.cpp`):

- Receives commands via USB serial (115200 baud)
- Commands:
 - `F xx` - Forward at xx% duty cycle (0-100)
 - `B xx` - Backward at xx% duty cycle (0-100)
 - `S` - Stop motor
- Pin connections:
 - `p13`: Motor forward signal
 - `p14`: Motor reverse signal
 - `p18`: Motor PWM voltage (AnalogOut)

Processing Visualizer (`motor_controller.pde`):

- Provides GUI with sliders and buttons
- Displays real-time motor status
- Sends commands to mbed via serial

Usage:

1. Flash `motor_controller.bin` to mbed (or compile `.cpp` yourself)
 2. Connect motor driver circuit
 3. Run `motor_controller.pde` in Processing
 4. Use GUI to test motor operation
-

2. Analog Input Visualizer (`analog_input_visualizer/`)

Purpose: Visualize 4 analog input channels in real-time

mbed Program (`analog_input_visualizer.cpp`):

- Reads 4 analog inputs at ~200 Hz
- Sends data via USB serial (115200 baud)
- Uses frame-based protocol: `[0xAA 0x55] + 4 bytes`
- Pin connections:
 - `p16`: Analog input CH1
 - `p17`: Analog input CH2
 - `p19`: Analog input CH3 (photo-interrupter 1)
 - `p20`: Analog input CH4 (photo-interrupter 2)

Processing Visualizer (`analog_input_visualizer.pde`):

- Displays 4-channel real-time waveforms
- Shows pendulum angle graphically
- Helps verify sensor alignment and operation

Usage:

1. Flash `analog_input_visualizer.bin` to mbed
 2. Connect analog sensors to inputs (p16-p20)
 3. Run `analog_input_visualizer.pde` in Processing
 4. Observe real-time sensor readings
-

3. Six-Channel Oscilloscope (`six_channel_oscilloscope/`)

Purpose: Monitor complete control system with 6 data channels

mbed Program (`six_channel_oscilloscope.cpp`):

- Implements full PD control loop at 2000 Hz
- Reads angle sensors (2 photo-interrupters)
- Controls motor based on feedback
- Sends 6 channels of data via USB serial (230400 baud)
- Uses frame-based protocol: `[0xAA 0x55] + 6 bytes`
- Data channels:
 1. Photo-interrupter 1 output
 2. Photo-interrupter 2 output
 3. Angle (difference of sensors)
 4. Angular velocity (ω)

5. Filtered angular velocity

6. Reference line (128)

Processing Visualizer (`six_channel_oscilloscope.pde`):

- Displays 6 channels in two 3-channel plots
- Upper plot: CH1 (Red), CH2 (Green), CH3 (Blue)
- Lower plot: CH4 (Red), CH5 (Green), CH6 (Blue)
- Essential for tuning control parameters

Usage:

1. Flash `six_channel_oscilloscope.bin` to mbed
2. Connect complete circuit (motor + sensors)
3. Run `six_channel_oscilloscope.pde` in Processing
4. Reset mbed to synchronize display
5. Tune PD gains while monitoring system response

How to Use

Option 1: Use Pre-compiled Binaries (Easiest)

1. Connect mbed LPC1768 to PC via USB
2. mbed appears as USB drive (e.g., `MBED` or `LPC1768`)
3. Drag and drop `.bin` file to mbed drive
4. Press reset button on mbed
5. Run corresponding `.pde` file in Processing

Option 2: Compile from Source

If you want to modify the programs or understand the code:

1. For mbed programs:

- Use [Keil Studio Cloud](#) (online IDE)
- Or use [mbed CLI](#) locally
- Import mbed-os library
- Compile `.cpp` file
- Download generated `.bin` file
- Flash to mbed

2. For Processing programs:

- Install [Processing](#)
- Open `.pde` file
- Click Run () button

- Select correct serial port in code if needed

Serial Communication Protocol

All tools use USB serial communication between mbed and PC:

Basic Principle

- **UART (Universal Asynchronous Receiver-Transmitter):** Serial protocol for byte-by-byte data transmission
- **USB-to-Serial:** mbed LPC1768 has built-in USB-to-serial converter
- **Pins:** `USBTX` and `USBRX` connect to USB interface
- **Baud Rate:** Communication speed (bits per second)
 - Motor Controller: 115200 bps
 - Analog Input Visualizer: 115200 bps
 - Oscilloscope: 230400 bps (higher for 2 kHz data rate)

Frame-Based Protocol (Visualizers)

To reliably transmit multiple data values, a frame-based protocol is used:

```
[Header: 0xAA 0x55] [Data Byte 1] [Data Byte 2] ... [Data Byte N]
```

- **Frame Header:** Two special bytes (`0xAA 0x55`) mark start of frame
- **Data Bytes:** Each sensor/variable value encoded as 0-255
- **Synchronization:** Processing searches for header pattern to align data stream

Text-Based Protocol (Motor Controller)

Simple ASCII commands for human readability:

- Commands sent as text strings (e.g., `"F 50\n"`)
- Easy to test with any serial terminal
- mbed parses commands using `sscanf()`

Pin Assignments Summary

Function	Pin	Type	Used By
Motor Forward	p13	DigitalOut	Motor Controller, Oscilloscope
Motor Reverse	p14	DigitalOut	Motor Controller, Oscilloscope
Motor Voltage	p18	AnalogOut	Motor Controller, Oscilloscope
Analog CH1	p16	AnalogIn	Analog Input Visualizer

Function	Pin	Type	Used By
Analog CH2	p17	AnalogIn	Analog Input Visualizer
Photo-interrupter 1	p19	AnalogIn	Analog Input Visualizer, Oscilloscope
Photo-interrupter 2	p20	AnalogIn	Analog Input Visualizer, Oscilloscope
USB Serial TX	USBTX	Serial	All tools
USB Serial RX	USBRX	Serial	All tools

Note: `p18` is the only AnalogOut pin on mbed LPC1768, so motor voltage control must use this pin.

Troubleshooting

mbed not recognized as USB drive

- Try different USB cable (some cables are power-only)
- Press reset button on mbed
- Check USB port on PC

Processing cannot find serial port

- Check port name in `.pde` file (usually auto-detected)
- On Windows: Look for `COM3`, `COM4`, etc.
- On macOS: Look for `/dev/tty.usbmodem*`
- On Linux: Look for `/dev/ttyACM*`
- Ensure no other program is using the port

No data displayed in Processing

- Reset mbed after starting Processing program
- Check baud rate matches between mbed and Processing
- Verify USB cable supports data (not just power)

Motor moves opposite direction

- Swap motor driver connections (`p13` ↔ `p14`)
- Or swap motor wire polarity

Sensor readings look wrong

- Check sensor power supply (3.3V, not 5V)
- Verify pull-up resistors are installed
- Adjust sensor alignment mechanically

- Check resistor values (typical: 1k Ω - 12k Ω)

Modifying the Programs

Changing Control Parameters (Oscilloscope)

Edit these `#define` values in `six_channel_oscilloscope.cpp`:

```
#define SAMP_FREQ 2000.0f // Sampling frequency (Hz)
#define VLIMIT 0.1f      // Motor voltage limit
#define KP 1.0f          // Proportional gain
#define KD 0.0f          // Derivative gain
```

After changing, recompile and flash to mbed.

Adding More Channels

The frame protocol can be extended:

1. Add more data bytes in mbed program after frame header
2. Update Processing to read additional bytes
3. Add visualization for new channels

Changing Serial Baud Rate

Must match in both mbed and Processing:

mbed: `pc.baud(115200);`

Processing: `myPort = new Serial(this, portName, 115200);`

Further Reading

- [mbed Serial API](#)
- [Processing Serial Library](#)
- [UART Communication Basics](#)

License

These tools are provided for educational purposes as part of the University of Tokyo wheeled inverted pendulum experiment.